

Infrared PR 648

Security Review

Solo review by:
R0bert, Lead Security Researcher

November 12, 2025

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	High Risk	4
3.1.1	Dead DVN in LayerZero default config bricks Bera → BNB lane	4
3.2	Low Risk	5
3.2.1	Missing BSC explorer entry	5
3.3	Informational	6
3.3.1	LZ default confirmations exceed chain finality	6
3.3.2	OFT owners must be multisigs	6
3.3.3	Verify script fails to enforce OFT invariants	6
3.3.4	setEnforcedOptions is never called	7

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

A security review is a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While the review endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that a security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Infrared simplifies interacting with Proof of Liquidity with liquid staking products such as iBGT and iBERA.

From Nov 6th to Nov 7th the security researchers conducted a review of [Infrared - PR 648](#) on commit hash [e6262ae5](#). A total of **6** issues were identified:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	1	1	0
Medium Risk	0	0	0
Low Risk	1	1	0
Gas Optimizations	0	0	0
Informational	4	2	2
Total	6	4	2

2.1 Scope

The security review had the following components in scope for [Infrared - PR 648](#) on commit hash [e6262ae5](#):

	script
	oft
	ConfigureOFTPeers.s.sol
	src
	periphery
	IROFT.sol
	IROFTAdapter.sol

3 Findings

3.1 High Risk

3.1.1 Dead DVN in LayerZero default config bricks Bera → BNB lane

Severity: High Risk

Context: `ConfigureOFTPeers.s.sol#L55-L60`, `ConfigureOFTPeers.s.sol#L84-L89`

Description: The Berachain->BNB Chain LayerZero pathway is still on the published defaults ([https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey\[0\]=bera&dstChainKey\[0\]=bsc](https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey[0]=bera&dstChainKey[0]=bsc)). Those defaults set `SendUln302/ReceiveUln302`, but the DVN list contains the placeholder `LZDeadDVN` (0x6788...842). No oracle signs under that address. When the adapter tries to deliver a packet, `EndpointV2` waits for verifications from `requiredDVNs`; with `LZDeadDVN` configured and the optional list empty, the challenge can never be satisfied and the message remains unverified.

The `0App` sees the path as "not verifiable", packets stay enqueued and users get hard failures for every Berachain->BNB transfer. This effectively bricks that bridge direction and strands any IR locked on Berachain after the first send attempt.

None of the LayerZero wiring calls described in the docs have been executed: `From0App.setEnforcedOptions()`, `EndpointV2.setSendLibrary()`, `EndpointV2.setReceiveLibrary()`, `EndpointV2.setReceiveLibraryTimeout()`, `EndpointV2.setConfig()` for both send and receive, nor `EndpointV2.setDelegate()`. Because everything falls back to Layer Zero configured defaults, the Berachain->BNB lane inherits the empty DVN set, so there is no verifier to advance packets and every bridge attempt fails. There is no default DVN set for Berachain->BNB.

See default configs below:

Source Berachain Destination BNB Chain:

- [[https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey%5B0\]=bera&dstChainKey\[0\]=bsc](https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey%5B0]=bera&dstChainKey[0]=bsc)]([https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey%5B0\]=bera&dstChainKey\[0\]=bsc](https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey%5B0]=bera&dstChainKey[0]=bsc))
- From/To: Berachain/BNB Chain.
- Send Library: `SendUln302` - 0xC39161c743D0307EB9BCc9FEF03eeb9Dc4802de7.
- Receive Library: `ReceiveUln302` - 0xB217266c3A98C8B2709Ee26836C98cf12f6cCEC1.
- DVN 1: `LZDeadDVN` 0x6788f52439ACA6BFF597d3eeC2DC9a44B8FEE842.
- DVN 2: None.
- Executor: LayerZero Labs 0x4208D6E27538189bB48E603D6123A94b8Abe0A0b.
- Send Confirmations: 20.
- Receive Confirmations: 20.

BNB Chain Destination Berachain:

- [[https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey%5B0\]=bsc&dstChainKey\[0\]=bera](https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey%5B0]=bsc&dstChainKey[0]=bera)]([https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey%5B0\]=bsc&dstChainKey\[0\]=bera](https://layerzeroscan.com/tools/defaults?version=V2&srcChainKey%5B0]=bsc&dstChainKey[0]=bera))
- From/To: BNB Chain/Berachain.
- Send Library: `SendUln302` - 0x9F8C645f2D0b2159767Bd6E0839DE4BE49e823DE.
- Receive Library: `ReceiveUln302` - 0xe1844c5D63a9543023008D332Bd3d2e6f1FE1043.
- DVN 1: Nethermind 0x31F748a368a893Bdb5aBB67ec95F232507601A73.
- DVN 2: LayerZero Labs 0xfD6865c841c2d64565562fCc7e05e619A30615f0.
- Executor: LayerZero Labs 0x3ebD570ed38B1b3b4BC886999fcF507e9D584859.
- Send Confirmations: 20.

- Receive Confirmations: 20.

Recommendation: Explicitly wire both pathways with real DVNs using the LayerZero CLI or manual `setConfig` (for example, reuse the Nethermind + LayerZero Labs committee used on the return leg). Run the wiring under multisig so `PeerSet` and `ConfigSet` transactions are reviewable, and only ship once `layerzeroscan` shows non-placeholder DVNs on both directions.

Manual alternative: call `setConfig` directly on `EndpointV2` for each direction with the encoded executor/DVN settings. Example:

```
SetConfigParam[] memory params = new SetConfigParam[](2);
params[0] = SetConfigParam({
    eid: EndpointId.BSC,
    configType: 1,
    config: abi.encode(ExecutorConfig({
        maxMessageSize: 10000,
        executor: 0x4208D6E27538189bB48E603D6123A94b8Abe0A0b
    }))
});
params[1] = SetConfigParam({
    eid: EndpointId.BSC,
    configType: 2,
    config: abi.encode(UlnConfig({
        confirmations: 20,
        requiredDVNCount: 0,
        optionalDVNCount: 2,
        optionalDVNThreshold: 2,
        requiredDVNs: new address[](0),
        optionalDVNs: new address[](2)
    }))
});
ILayerZeroEndpointV2(endpoint).setConfig(address(adapter),
    ↪ 0xC39161c743D0307EB9BCc9FEF03eeb9Dc4802de7, params);
```

Repeat on BNB Chain with its executor/DVN addresses. This mirrors the doc's "Adding sendConfig/receiveConfig" procedure.

Infrared Finance: Fixed in commit [04c2c68](#)

R0bert: Fix verified.

3.2 Low Risk

3.2.1 Missing BSC explorer entry

Severity: Low Risk

Context: [foundry.toml#L67-L69](#)

Description: `foundry.toml` only registers explorer configs for Berachain and Bepolia, so when the developers follow the scripts' instructions and run `forge script/deploy/DeployIROFT.s.sol --rpc-url $BINANCE_RPC --broadcast --verify`, Foundry emits a warning and silently skips verification on BNB Chain because no `[etherscan]` entry matches that network. As a result the `IROFT` contracts end up deployed but unverified on BscScan, hindering transparency and making it harder for downstream tooling to interact with the bytecode.

Recommendation: Add a BNB explorer config and document the corresponding API key: `bsc = { key = "${BSCSCAN_API_KEY}", url = "https://api.bscscan.com/api" }`. Ensure operators export `BSCSCAN_API_KEY` so that `--verify` uploads succeed on Binance Chain.

Infrared Finance: Fixed in commit [3361b27](#)

R0bert: Fix verified.

3.3 Informational

3.3.1 LZ default confirmations exceed chain finality

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: LayerZero's default `SendUln302/ReceiveUln302` configs assume 20 block confirmations before a packet is considered final. That value is excessive for both networks in this bridge. Berachain advertises single-slot finality, meaning blocks are finalized in the same slot they are proposed (see <https://docs.berachain.com/learn/help/glossary#single-slot-finality>). BNB Chain's BEP-126 "Fast Finality" upgrade reduces finalization to roughly 2.5 blocks (~7.5 seconds) instead of the legacy 15-block (~45 seconds) window (see <https://www.bnbchain.org/en/blog/what-is-finality-in-blockchain>). Leaving the LayerZero configs at 20 confirmations introduces unnecessary latency and ties up liquidity for several multiples of the actual finality window, hurting UX and capital efficiency.

Recommendation: Update the LayerZero send/receive configs for both the adapter and the OFT to reflect realistic confirmation counts (e.g., 2-3 blocks on Berachain and 3 blocks on BSC) instead of the default 20. Execute these changes via the endpoint's `setConfig` calls (or the LayerZero CLI) so packets settle as soon as the underlying chain finalizes, while keeping a safety margin that matches each chain's documented guarantees.

Infrared Finance: Fixed in commit [04c2c68](#)

R0bert: Fix verified.

3.3.2 OFT owners must be multisigs

Severity: Informational

Context: [DeployIROFTAdapter.s.sol#L28](#), [DeployIROFT.s.sol#L26](#)

Description: Both `IROFT` and `IROFTAdapter` set their `Ownable` owner to the same address they pass into the LayerZero delegate slot. That owner/delegate can call `setPeer`, rotate DVNs, change send/receive libraries, alter enforced options, or completely halt cross-chain messaging via LayerZero's config surface. If a single EOA controls those privileges and it is lost or compromised, an attacker can divert all bridged liquidity, mint arbitrary amounts on remote chains, or permanently brick the bridge by pointing to malicious peers. Because these scripts encourage providing `OFT_OWNER/ADAPTER_OWNER` from env vars without clarification, teams might deploy with a hot wallet and never migrate, leaving critical controls in a single key's custody.

Recommendation: Document and enforce that both deployment scripts must be executed with multisig addresses for `OFT_OWNER` and `ADAPTER_OWNER`. Ideally, add validation to the scripts (e.g., require the owner address to be an expected Safe address) and create follow-up runbooks so that ownership delegation is never assigned to an EOA.

Infrared Finance: Fixed in [3361b27](#)

R0bert: Fix verified.

3.3.3 Verify script fails to enforce OFT invariants

Severity: Informational

Context: [VerifyOFTSetup.s.sol#L18](#)

Description: `script/oft/VerifyOFTSetup.s.sol` is marketed as a verification script but it only prints state instead of enforcing it. When peers are misconfigured the script still exits successfully because lines 53-58 and 105-109 only log "INCORRECT" and never revert, so CI or deploy operators receive a green run even though the adapter and OFT point to the wrong counterpart. The same happens for every other parameter: lines 33-44 and 61-64 pull `token()`, `owner()` and `endpoint()` but never compare them to known addresses, while lines 85-116 dump metadata like `name`, `symbol`, `decimals`, `totalSupply`, `owner` and `endpoint` without any validation. Finally, the helper instructions at lines 15-16 also reference the wrong file path (`script/VerifyOFTSetup.s.sol`), so the documented command fails. Impact: misconfigured OFT deployments can remain undetected.

Recommendation: Update the script to turn every log-based check into a strict assertion, e.g. `require(peerAddress == expectedOftAddress, "wrong peer")` and pass the expected token, owner, and endpoint addresses in through env vars so they can be compared against `adapter.token()`, `adapter.owner()`, `adapter.endpoint()`, and their OFT counterparts. Iterate through all relevant peer EIDs to ensure every mapping points to the intended remote, guard metadata reads with try/catch or omit them if the token lacks those interfaces and fix the usage comment to reference `script/oft/VerifyOFTSetup.s.sol` so operators run the right command.

Infrared Finance: Acknowledged. Our current operational flow is to run `script/oft/VerifyOFTSetup.s.sol` manually and inspect the console output. We are treating the log messages as the verification step instead of hard assertions.

Robert: Acknowledged.

3.3.4 `setEnforcedOptions` is never called

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: `IROFTAdapter` and `IROFT` inherit `OFT(Core)` which embeds `OAppOptionsType3`. That contract exposes `setEnforcedOptions`, but none of the deployment scripts, Makefile target, or docs ever call it. When `enforcedOptions[_eid][_msgType]` is empty, `combineOptions` simply returns the caller-provided bytes and the LayerZero executor honors whatever gas/value budget the user encoded. A user can craft `extraOptions` with `gasLimit = 0` and negligible native value, so `quoteSend` charges almost nothing and `lzReceive` immediately runs out of gas on the destination. The packet then enters the "failed" queue until someone manually retries it and subsidizes the missing gas. An attacker can send dust transfers with zero gas limits; each packet will then sit in the failed queue until someone manually retries it. Merely informative as this is simply a self-griefing vector.

Recommendation: After wiring peers, immediately call `setEnforcedOptions` for every `(eid, msgType)` pair the OFT supports (at least `SEND` and `SEND_AND_CALL`). Encode a minimum executor gas limit and `msg.value` using `OptionsBuilder`'s `addExecutorLzReceiveOption` then lock those values via the owner account so every sender must prepay sufficient resources.

Reference: <https://docs.layerzero.network/v2/developers/evm/oft/quickstart#execution-options-for-composed-messages>.

Infrared Finance: Acknowledged.

Robert: Acknowledged.